

지 지형 — 개발 세계는 어떻게 나뉘나

골 골목 — 폴더는 왜 이렇게 갈라졌나

문 문패 — 파일들은 무슨 신분인가

만든다 이해한다

구조를 그린다 — 개발 세계의 지도 · 오늘은 아무것도 만들지 않습니다. 대신 전부 이해합니다

10차엔 소스를 감췄습니다 — 오늘은 다 펼칩니다

여러분은 관찰로 뽑힌 반입니다. 이제 겉이 아니라 속을 봅니다.

10차 — 소스를 감췄다

- 실행 파일 하나만 드렸다 (소스 비공개)
- 겉과 결과물만으로 속을 추론했다
- 측정한 것 = 사용자로서의 관찰력

오늘 — 속 지도를 펼친다

- 교재 = 여러분이 매주 점수 받던 그 채점기
- 폴더 하나하나, 파일 하나하나 — 왜 있는지
- 얻는 것 = 개발 세계를 읽는 지도

진단받던 그 도구가, **오늘의 교재**입니다.

개발 지식 시리즈 — 내 앱 설계 3종 세트

격주로 초급반과 교차합니다. 지금 잡힌 계획은 3회 — 이후는 진행을 보고 이어집니다.

12

구조를 그린다

웹 작동 원리 · 폴더 구조 · 좋은 구조 → 숙제: 내 앱 구조도

14

화면을 그린다

HTML·CSS·JS · 컴포넌트 · UX/UI → 숙제: 내 앱 화면도

16

데이터를 그린다

DB · 테이블 · 보안 → 숙제: 내 앱 데이터도

세 장을 모으면 「내 앱 설계 3종 세트」 — 그다음에, 진짜로 만듭니다.

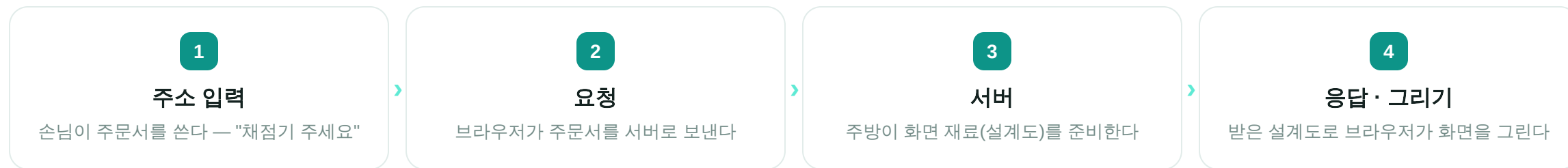
매 회차는 **직전 숙제 리뷰**로 시작합니다 — 오늘의 숙제가 14차의 첫 재료.

"모든 파일에는 **자리**가 있고,
그 자리에는 **이유**가 있다"

여러분은 이미 전부 해봤습니다 — 만들고, 배포하고, 키까지 숨겨봤죠.
이미 다 해봤습니다. 오늘은 이름을 배웁니다.

주소창의 여정 — 엔터 후 3초 동안 생긴 일

채점기 주소를 치고 엔터를 눌렀을 때, 화면이 뜨기까지.



서버가 보내는 건 완성된 그림이 아니라 **설계도** — 그림은 **여러분 손안의 브라우저**가 그립니다.

인터넷의 모든 것이 이 **요청** → **응답** 한 왕복의 반복입니다 — 검색도, 영상도, 채점도.

브라우저와 서버 — 두 대의 컴퓨터

방금 여정의 양 끝 — 사실 둘 다 그냥 컴퓨터(에서 도는 프로그램)입니다.

브라우저 — 내 손안의 컴퓨터에서

- 주소를 요청으로 바꿔 보내는 손님
- 받은 설계도로 화면을 그리는 화가
- 크롬·사파리·엣지 — 이름만 다른 같은 직업

서버 — 어딘가의 컴퓨터에서

- 요청을 기다렸다 응답하는 일을 맡은 컴퓨터
- 특수 장비 X — 여러분 노트북도 서버가 될 수 있다
- 단, 노트북을 덮으면 가계도 닫힌다 → 그래서 안 꺼지는 컴퓨터를 빌린다 (Railway)

서버 = 신비한 기계 X — 응답하는 일을 맡은, 안 꺼지는 컴퓨터 ✓

개발 세계의 큰 지형 — 4겹의 실명

데이터 · 동작 · 화면 · 연결 — 그 네 칸을 개발 세계는 이렇게 부릅니다. 지금부터 하나씩 자세히.

화면 →

프론트엔드

손님 앞의 **홀** — 브라우저에서 실행되는 전부

동작 →

백엔드

손님이 못 들어가는 **주방 안쪽** — 서버에서 실행

데이터 →

DB

꺾다 커도 남는 **창고** — 표로 저장하는 전문 프로그램

연결 →

API

프로그램끼리의 **주문 창구** — 정해진 양식의 대화

이미 다 해봤습니다. **오늘은 이름을 배웁니다.** — 지금부터 네 지형을 한 곳씩 밟습니다.

프론트엔드 — **홀** · 사용자의 기기에서 실행되는 전부

정의 어디서 도나

여러분의 브라우저 안에서 실행된다 — 버튼·글씨·색·애니메이션, 눈에 보이고 손에 닿는 전부

재료 삼형제 예고

뼈대(HTML) · 옷(CSS) · 움직임(JS) — **14차의 주인공들**, 오늘은 이름만

구분 UI vs UX

UI = 생김새(버튼 색·배치) / **UX = 경험**(몇 번 눌러야 하나, 헛갈리지 않나). "예쁘다"는 UI, "채점하자마자 등록 버튼이 보여 편하다"는 UX

기억할 것 — 프론트엔드 코드는 **전부 사용자에게 보입니다** (브라우저에 내려받아 실행되니까).
그래서 **비밀은 여기 못 듭니다** — 백엔드가 필요한 첫 번째 이유.

백엔드 — 주방 안쪽 · 왜 손님에게 안 보여주나

서버에서 실행되는 것 — 계산하고, 판단하고, 저장을 지시한다. 숨기는 데는 세 가지 이유가 있습니다.

이유 ①

비밀

API 키·채점 기준은 손님 손에 들어가면 안 된다 — 프론트는 전부 보이니까, 비밀은 서버에만

이유 ②

신뢰

점수 계산을 손님 쪽에서 하면? 조작할 수 있다. 심판은 주방에 있어야 한다

이유 ③

힘

무거운 계산은 좋은 컴퓨터(서버)가 — 손님 폰이 약해도 결과만 받으면 된다

채점기가 여러분 폰에서 채점하지 않는 이유 — 셋 다입니다.

DB — **참고** · 왜 그냥 파일이 아닌가

데이터를 표(테이블)로 저장하는 전문 프로그램 — 행 = 기록 한 건, 열 = 항목.

"엑셀 파일에 쓰면 안 되나요?"

- 두 명만 동시에 써도 꼬인다
- 수만 건에서 찾기가 느리다
- "점수 칸에 글자" 같은 실수를 못 막는다

DB — 참고 전문가

- 동시에 **100명**이 제출해도 안 깨진다
- 수만 건 중 "닉네임 A의 최고점"을 **즉시** 찾는다
- 규칙 위반을 **참고**가 막아준다

리더보드 = 표 하나 — 닉네임 | 점수 | 제출 시각 . 앱을 꺼도, 서버가 재시작돼도 **참고는 남는다**. (참고 속은 16차에서 통째로)

API — 주문 창구 · 프로그램이 프로그램에게 말 거는 법

사람은 화면(UI)으로 앱과 대화하고 — 프로그램은 API로 대화합니다. 정해진 양식으로 묻고, 정해진 형식으로 답합니다.

양식 주문서 예시

요청: "이 프롬프트 채점해줘 (텍스트)" → 응답: "점수표 (정해진 형식)" — **양식이 정해져 있어 기계끼리 통한다**

세상 어디에나

지도 앱이 길을 아는 것, 날씨 앱이 기온을 아는 것 — 전부 **남의 API를 불러 쓰는 것**

채점기 두 군데

① 브라우저 → 채점기 서버의 `/api/score` (우리가 만든 창구) ② 채점기 서버 → **Claude API** (우리 서버가 손님이 되어 채점을 주문)

양식이 정해져 있으니 — **서로 모르는 프로그램끼리도 협업**합니다. 그게 연결의 힘.

채점기 하나에 다 있다

9차의 5요소 기억나시죠 — 몸·기억·무대·창고·맥박. 오늘은 그 '몸' 속으로 들어갑니다.

Next.js 몸

출과 주방을 한 몸에 — 화면(프론트엔드)과 동작(백엔드)이 한 폴더에 사는 조립 키트

Supabase 창고

DB — 리더보드 점수가 사는 곳. **우리 폴더 밖**, 다른 회사 서버에 있다. 우리가 가진 건 열쇠와 주소뿐 — 그래서 '연결'이 필요

Railway 무대

배포 — 조리된 앱이 24시간 서 있는 곳. push하면 맥박처럼 자동으로 올라간다

오늘 지도의 대상은 이 '**몸**' 폴더 — 여러분이 매주 쓰던 채점기의 속입니다. (잠시 후 10분 휴식 뒤, 폴더를 엽니다)

루트 — 폴더를 열면 처음 보이는 것들

프롬프트 채점/

- └ **package.json** ← 재료 목록 — 이름·부품·명령어
- └ 설정 파일들 ← 주방 세팅 (tsconfig 등 4개)
- └ **.env.local** ← 열쇠 서랍 🗝️ (git엔 절대 안 올라감)
- └ .gitignore ← 문지기 — 못 올라갈 것들의 명단
- └ **README.md** · **CLAUDE.md** ← 문패 2장 — 사람용 · AI용
- └ docs/ · public/ ← 문서 · 그대로 내보내는 것들
- └ **node_modules/** ← 창고 — 내가 안 만듦 (제일 무거움)
- └ **src/** ← 내가 만든 것의 전부

내가 만든 건 **절반뿐** — 나머지는 도구와 관례가 만들었습니다. 그 정체를 지금 밝힙니다.

내가 안 만든 파일들의 정체 — 요리로 읽기

목록 package.json

재료 목록 — 앱 이름, 필요한 부품(패키지), 실행 명령. 이 목록만 있으면 누구든 같은 환경을 재현한다

참고 node_modules

장 봐온 참고 — `npm install` 이 목록을 보고 가게(npm)에서 부품을 사다 채운다. 수만 개 파일, 제일 무겁다

조리 빌드

쓰기 좋은 코드를 돌리기 좋은 형태로 변환·압축 — `npm run build` → .next 폴더

열쇠 .env

열쇠 서랍 — 비밀값을 코드와 분리. 코드에 직접 쓰면 GitHub에 올라가는 순간 전 세계 공개

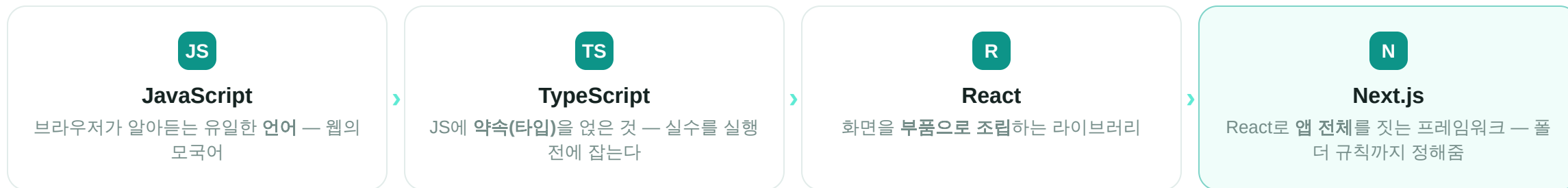
문지기 .gitignore

비밀(.env) · 재생 가능한 것(참고·조리 결과) · 개인 설정 — 셋을 GitHub에서 막는다

Q. 그럼 — **node_modules**를 지우면 앱은 죽을까요? (손 들어봅시다)

그 많던 이름들 — 러시아 인형 · "그거 다 Next.js 아니었어?"

언어 = 말 · 라이브러리 = 내가 부르는 부품 상자 · 프레임워크 = 나를 끼워 넣는 조립 키트



채점기는 **Next.js**로 지은 앱이고, 속은 **React** 부품이고, 부품은 **TypeScript**로 쓰였고, TS는 결국 **JavaScript**입니다.

"폴더가 곧 주소" 규칙도 Next.js가 정한 것 — 잠시 후 만납니다. 깊은 얘기는 14차에서.

문패 2장 — 사람용과 AI용

6차에서 만들었던 그 두 파일 — 문서도 구조의 일부입니다.

README.md — 사람용 문패

- 뭐 하는 앱인지, 어떻게 켜는지
- 폴더를 연 사람이 **제일** 먼저 읽는 파일
- 대문자 이름 = "나부터 읽어라"(READ ME!)라는 관례
- 6개월 뒤의 나도 '처음 온 사람'이다

CLAUDE.md — AI용 문패

- 규칙 · **합정** · 히스토리를 AI에게
- AI는 매번 **처음 출근한 직원** — 수칙이 없으면 같은 사고를 반복한다
- 문서가 곧 **AI의 기억**
- 6차에서 우리가 직접 만든 그 파일

채점기의 CLAUDE.md는 **39KB — A4 30장**. 11개 강의의 역사가 전부 적혀 있습니다.

src — 내가 만든 것의 다섯 골목

src/

└ app/	← 주소가 있는 것들의 동네	화면	연결
└ components/	← 화면 부품 26개 - 레고	화면	
└ lib/	← 동작 - 채점 기준·세션 처리	동작	
└ types/	← 계약 - 주고받는 것의 약속	계약	
└ data/	← content.ts 한 파일 = 225KB	데이터	

한 폴더가 두 겹에 걸치기도 합니다 — **지도는 단순화**입니다. 실물은 경계가 흐려요.
그래도 지도 없이 다니는 것보다 백 배 낫습니다. (숙제에서 태그 두 개 붙여도 됩니다)

폴더가 곧 주소다

app 골목의 규칙 (Next.js가 정한 관례) — 폴더 경로가 그대로 URL이 됩니다.

/	랜딩	app/page.tsx	— 강의 소개 첫 화면
/practice	실습	app/practice/	— 채점 연습하던 그 페이지
/play/ABCD	세션	app/play/[code]/	— 대괄호 = 변하는 주소 (세션 코드마다 다른 방)
/api/score	창구	app/api/score/	— 화면 없는 골목 . 주소는 있는데 보이는 건 없다 — 채점 백엔드

Q. 그럼 채점기의 **화면(주소)**은 모두 몇 개일까요? — 매주 쓰신 앱입니다. (폰으로 눌러봐도 됩니다)

같은 원리, 다른 생김새

파이썬 프로젝트를 열어도 길을 잃지 않는 법 — 이름만 바꿉니다.

JavaScript — 우리 채점기

- `package.json` — 재료 목록
- `node_modules/` — 참고
- `src/` — 소스 (내가 만든 것)

Python — 이름만 다르다

- `requirements.txt` — 재료 목록
- `venv/` — 참고
- `main.py` · `src/` — 소스

이름은 달라도 **자리는 같다** — 재료 목록 · 참고 · 소스. 오늘 배운 건 Next.js의 지도가 아니라 **자리를 읽는 법**입니다.

좋은 구조의 원칙 5

원칙 ①

한 폴더 = 한 책임

이유를 한 줄로 못 쓰는 폴더는 없어야 할 폴더다

원칙 ②

화면과 동작을 섞지 않는다

섞으면 로직 고치다 화면이 깨진다 — 서로 인질이 된다

원칙 ③

주고받는 것은 계약으로

types/ — 점수의 생김새를 약속. 2차 I/O 계약의 코드판

원칙 ④ ★

데이터와 코드를 섞지 않는다

강의 내용은 전부 content.ts 한 파일 — 코드 수술 없이 강의가 추가된다

원칙 ⑤

문패를 단다

사람용(README)과 AI용(CLAUDE.md) — 문서도 구조다

속제에서

이 다섯이 채점 기준

여러분의 구조도는 이 원칙으로 스스로 검사합니다

증거 — 이 채점기에 강의가 **11개** 추가되는 동안, 대부분의 회차에서 고친 파일은 **content.ts 하나**였습니다.

채점 버튼 하나의 여정 — 수백 번 누른 그 버튼



열쇠(.env)는 **3번 — 서버 쪽에서만** 열립니다. 브라우저에 뒀다면 여러분 모두가 제 API 키를 가져갈 수 있었어요. (원리는 16차에서)

아까는 **주소**로 들어왔고, 이번엔 **버튼**으로 들어왔습니다 — 이 여정이 그려지면, 오늘 지도는 완성입니다.

속제 — 내 앱 폴더 구조 설계

만들고 싶은 앱 하나를 정해, 오늘 배운 원칙대로 폴더 트리를 설계지에 그려웁니다. 종이와 펜이면 충분합니다.

합격 기준 ①

최상위 폴더 3~6개

더 많으면 책임이 안 갈라진 것

합격 기준 ②

모든 폴더에 "왜" 한 줄

이유를 못 쓰는 폴더는 지운다

합격 기준 ③

4겹 태그

폴더마다 데·동·화·연 중 하나 이상 (두 개도 OK)

보너스 — "내가 안 만들 파일" 자리 1개 이상 비워두기 (node_modules처럼 도구가 만들어줄 자리)

14차는 이 종이로 시작합니다 — 반드시 지참! 다음 시간엔 이 구조 위에 화면을 그립니다.

모든 파일에는 자리가 있고, 그 자리에는 **이유**가 있다

오늘 처음으로 — 지도를 들고 개발 세계에 들어왔습니다 · 설계지 꼭 챙겨 오세요
수고하셨습니다!

- 부록 · 질문이 나오면 이 장을 띄우세요

전체 지도 한 장 — 오늘 배운 골목 전부

프롭포트 채점/

- └ **package.json** 재료 목록 (+lock = 영수증) · 설정 파일 4종 — 주방 세팅
- └ **.env.local** 열쇠 서랍 🔑 · **.gitignore** 문지기 — 비밀·재생 가능한 것을 막음
- └ **README.md** 사람용 문패 · **CLAUDE.md** AI용 문패 (39KB)
- └ docs/ 문서 · public/ 그대로 내보내는 것
- └ **node_modules/** 창고 — 지워도 npm install로 재생 · **.next/** 조리 결과 — 빌드 산출물
- └ **src/** — 내가 만든 것의 전부
 - └ **app/** 주소 동네 — 페이지 + api 창구 **화면** **연결** 폴더가 곧 주소 · [code]=변하는 주소
 - └ **components/** 화면 부품 26개 — 레고 **화면**
 - └ **lib/** 동작 — 채점 기준·세션 처리 **동작**
 - └ **types/** 계약 — 주고받는 것의 약속
 - └ **data/** content.ts 225KB — 콘텐츠 데이터 **데이터**

채점 버튼의 여정: 버튼(components) → /api/score → 채점 기준+Claude(lib·🔑) → 결과 카드 → Supabase(폴더 밖 창고) → 리더보드
원칙 5 — 한 폴더 한 책임 · 화면·동작 분리 · 계약 따로 · 데이터·코드 분리 · 문패